PAGE NO.....
DATE / /

Window Functions

The `Row_number()` function in SQL is a window functions that assigns a unique sequential integer to rows within a partition of a resultset. The numbering starts at 1 for the 1st row in each partition. This is particularly useful when you want to rank or filter rows based on specific criteria, such as finding the most expensive product sold by each vendor.

Key points about `ROW_NUMBER()`:

1. Partitioning: The `Partition by` clause divides the result set into partitions (groups) based on one or more columns. The `ROW_NUMBER()` function is applied separately to each partition.

2. Ordering: The `ORDER BY` clause within the `ROW_NUMBER()` functions determines the order in which are numbered within each partition.

3. Subqueries: To filter based on the row number (e.g. selecting only the top-ranked row per partition) you must use a subquery because window functions like `ROW_NUMBER()` are evaluated after the `where` clause

- vandor-invantory

invantory_id	vandor_id	product_id	market_date	original_price
1	101	1001	2023-10-01	10.50
2	101	1002	2023-10-01	15.75
3	101	1003	2023-10-02	20.00
4	102	1004	2023-10-01	12.00
5	102	1005	2023-10-02	18.50
6	102	1006	2023-10-03	22.00
7	103	1007	2023-10-01	9.99
8	103	1008	2023-10-02	14.50
9	103	1009	2023-10-03	19.99

Int primary key | Int | Int | Date | Decimal (10,2)

Quary to rank product by price per vandor

```
Select
    vandor_id,
    market_date,
    product_id,
    original_price,
    Row Number() over ( partition by vandor_id order by
                        original_price desc) As price_rank
From vandor_invantory
Order by vandor_id, price_rank;
```

PAGE NO.....
DATE / /

1) ROW-NUMBER()

↳ This is the window function that assigns a unique sequential integer to each row within a partition.

↳ The numbering starts at 1 for the 1st row in each partitions.

2) OVER:

↳ The Over clause defines the window or set of rows over which the ROW-NUMBER() function operate.

↳ It specifies how to partitions and order the rows.

3) Partition by vendor_id.

↳ The PARTITION BY clause divides the result set into partitions (groups) based on the value of vendor_id

↳ Each partition is treated as a separate group, and the ROW_NUMBER() function is applied independently to each partition.

↳ For example, if there are 3 vendors, the row will be split into 3 partitions, on for each vendor

4) ORDER BY original_price DESC.

→ The ORDER_BY clause specifies the order in which rows are numbered within each partition.

→ In this case, rows are sorted by original_price in descending order (highest price first)

PAGE NO.....
DATE / /

→ The row with the highest original_price in each partition will be assigned price_rank = 1.

5) As price_rank:

→ This assigns an alias (price_rank) to the output of the ROW_NUMBER() function.

→ The alias can be used in the SELECT statement or in subsequent filtering (e.g. in a subquery)

How it works

1. Partitioning:

→ The row are divided into partitions based on vendor_id

- Partition 1: vendor_id = 101
- Partition 2: vendor_id = 102
- Partition 3: vendor_id = 103

2. Ordering:

- within each partition, rows are sorted by original_price in descending order

↳ for vendor_id = 101:

- Row 3: original_price = 20. (rank 1)
- Row 2: original_price = 15.75 (rank 2)
- Row 1: original_price = 10.60 (rank 3)

↳ for vendor_id = 102:

- Row 6: original_price = 22.00 (rank 1)
- Row 5: original_prices = 18.50 (rank 2)
- Row 4: original_price = 12.00 (rank 3)

→ For vendor_id = 103:

- Row 9: original_price = 19.19 (rank 1).
- Row 8: original_price = 14.50 (rank 2).
- Row 9: original_price = 9.99 (rank 3).

3) Row Numbering

- The Row_Number() function assigns a unique number to each row within its partitions.

→ For vendor_id = 101:

- Row 3: price_rank = 1.
- Row 2: price_rank = 2.
- Row 1: price_rank = 3.

→ For vendor_id = 102:

- Row 6: price_rank = 1.
- Row 5: price_rank = 2.
- Row 4: price_rank = 3.

4. Final Output:

vendor_id	market_date	product_id	original_price	price_rank
101	2023-10-10	1003	20.00	1
101	2023-10-01	1002	15.75	2
101	2023-10-01	1001	10.50	3
102	2023-10-03	1006	22.00	1
102	2023-10-02	1005	18.50	2
102	2023-10-01	1004	12.00	3
103	2023-10-03	1009	19.99	1
103	2023-10-02	1008	14.500	2
103	2023-10-01	1007	9.99	3

PAGE NO.....
DATE / /

--> Query to find the most expensive product per vendor.

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price
From (
    SELECT
        vendor_id,
        market_date,
        product_id,
        original_price,
        ROW_NUMBER() OVER(PARTITION BY vendor_id
            ORDER BY Original_price Desc) As pricerank
From vendor_inventory
) As ranked_inventory
Where price_rank = 1.
Order by vendor_id;
```

vendor_id	market_date	product_id	original_price
101	2023-10-02	1003	20.00
102	2023-10-03	1006	20.00
103	2023-10-03	1009	19.99

PAGE NO.....
DATE / /

Explanation 1

(1) Row_Number() Over (Partition by vendor_id Order by original_price Desc) As price_rank:

↳ This window function generates a row number (price_rank) for each product within a partition defined by vendor_id.

↳ Partition by vendor_id: Groups rows by each vendor similar to a Group-by but without aggregating.

↳ Order By original_price Desc: Orders the product within each vendor partition from the highest price to the lowest.

↳ As price_rank: Assign the row number to a column named price_rank

↳ As a result, the product with the highest price for each vendor gets price_rank = 1

(2) As ranked_inventory

→ The result of the inner query is given an alias ranked_inventory treating it like a table that can be referenced in the outer query.

(3) Where price_rank = 1

→ Filters the rows to keep only the record where price_rank is equal to 1

→ This ensures that only the highest-priced product for each vendor is included in the final result.

PAGE NO.....
DATE / /

(4) ORDER By vendor_id.

- sort the output rows by vendor_id in ascending order to organise the results neatly.

What is the Role of Functions?

→ Imagine you are at a sport event, and there are different races happening for different age groups (kids, teenagers (adults). Your task is to rank the runners based on their finishing times.

→ Problem: How do you assign rank without mixing the age groups but still keeping track of every participant's details?
This is where OVER Functions come in.

Suppose

Runner	Age Group	Finish time
John	Teenager	12 seconds
Alice	Teenager	10 seconds
Bob	Teenager	11 seconds
Mike	Adult	13 seconds
Sam	Adult	12 seconds.

Row_Number() Over(Partition By Age Group ORDER BY FinishTime)

How Does it Work?

1) PARTITION BY AgeGroup:

→ Think of it like separating races into groups (kids, teenagers, adults)

- We only compare times within each group.

2) ORDER BY FinishTime.

→ sorts the runners by their finish time in ascending order.

3) ROW_NUMBER().

→ Assigns rank numbers starting from 1 within each partition (group).

Why Not Use Group By Instead?

If you used [GROUP BY], you could only get summarized data (like the best finish time for each age group). You lose row details about each runner.

The [OVER] function lets you keep all rows and assign ranks or perform other calculations per group.

PAGE NO.....
DATE / /

RANK AND DENSE_RANK Window functions

The RANK and DENSE_RANK functions are window functions in SQL that help assigns ranks to rows within partitions of a dataset, based on the ordering of a specific column. These functions are particularly useful when analysing ranked data, where multiple rows may share the same ranking value.

Both functions have the same syntax as ROW_NUMBER(), but their ranking logic differs when there are ties.

Rank Functions

- The Rank() function assigns the same rank to rows with the same values.
- However, if two rows share the same rank, the next rank will be skipped.
- This behavior results in gaps in the ranking.

Dense_rank function

- The DENSE_Rank() function also assigns the same rank to rows with the same value.
- Unlike Rank, it does not skip any rank values, ensuring consecutive ranks even in the presence of ties.

Row_Number function (For comparison)

→ The ROW_Number() function assigns a unique rank to each row, even if multiple rows have the same value. There are no ties.

(All three functions assign ranks or row numbers based on the values in the columns specified in the ~~ORDER BY~~ clause.)

PAGE NO.....
DATE / /

Rank, Dense_Rank and Row_Number in SQL

The SQL window functions Rank, Dense_Rank, and Row_Number are used to assigned rank values to rows within partitions of a dataset. They are valuable in scenarios that require ranking or ordering records based on specific criteria.

Key Differences:

- RANK: Assigns the same rank to rows with the same value but skips rank for ties. If multiple rows have the same value, they get the same rank. But the next rank is
- ~~Dense_Rank~~: skipped. Think of it like a school competition where two students tie for 1st place, so the next student gets 3rd place.

- Dense_Rank: Like Rank, but without skipping numbers If two student tie for 1st place, the next student gets 2nd place.

- Row_Number: Every row gets a unique number even if the values are identical. No ties allowed like assigning seat numbers in a theater.

database: window_functions. table_name: vendor_inventory1

vendor_id	market_date	product_id	original_price
1	2025-02-01	101	15.00
1	2025-02-01	102	15.00
1	2025-02-01	103	12.00
2	2025-02-01	201	20.00
2	2025-02-01	202	18.00
2	2025-02-01	203	18.00
Int	Date	Int	Decimal

PAGE NO.....
DATE / /

Rank() Function

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
Rank() Over ( Partition by vendor_id Order by original_price
        desc) As price_rank
From vendor_inventory1
Order by vendor_id, price_rank;
```

output

vendor_id	market_date	product_id	original_price
1	2025-02-01	101	15.00
1	2025-02-01	102	15.00
1	2025-02-01	103	12.00
2	2025-02-01	201	20.00
2	2025-02-01	202	18.00
2	2025-02-01	203	18.00

Explanation:

- Rows with the same price get the same rank.
- The next rank is skipped when there are ties.

Dense_Rank()

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
Dense_Rank() Over ( Partition by vendor_id Order by original_price
        desc) As price_rank
From vendor_inventory1
Order by vendor_id, price_rank;
```

PAGE NO.
DATE / /

Rank() Function

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
Rank() Over ( Partition by vendor_id Order by original_price
              desc ) As price_rank
From vendor_inventory1
Order by vendor_id, price_rank;
```

output

vendor_id	market_date	product_id	original_price
1	2025-02-01	101	15.00
1	2025-02-01	102	15.00
1	2025-02-01	103	12.00
2	2025-02-01	201	20.00
2	2025-02-01	202	18.00
2	2025-02-01	203	18.00

Explanation:

- Rows with the same price get the same rank.
- The next rank is skipped when there are ties.

Dense_Rank()

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
Dense_Rank() Over ( Partition by vendor_id Order by Original_price
                    desc) As price_rank
from vendor_inventory1
Order by vendor_id, price_rank;
```

PAGE NO.....
DATE / /

Vendor_id	market-date	product-id	Original-price	price-rank
1	2025-02-01	101	15.00	1
1	2025-02-01	102	15.00	1
1	2025-02-01	103	12.00	2
2	2025-02-01	201	20.00	1
2	2025-02-01	202	18.00	2
2	2025-02-01	203	18.00	2

Explanation

- Similar to RANK, but Dense-Rank does not skip ranks.

price-rank · Row-number function

```
1       Select
1         vendor_id,
3         market-date,
1         product-id,
2         original-price,
2.        Row_Number() Over( Partition by vendor_id Order by
                    original-price desc) As price-rank
          From vendor_inventory1
          Order by vendor_id, price-rank;
```

Vendor_id	market-date	product-id	Original-price	price-rank
1	2025-02-01	101	15.00	1
1	2025-02-01	102	15.00	2
1	2025-02-01	103	12.00	3
2	2025-02-01	201	20.00	1
2	2025-02-01	202	18.00	2
2	2025-02-01	203	18.00	3

Explanations:

- Always assigns a unique rank to each row.
- No ties are allowed.

PAGE NO.....
DATE / /

NTILE

1) What it does:

→ The NTILE function divides rows into a specified number of groups (n) and assigns a group number to each row.

→ It is useful for splitting data into equal (or nearly equal) buckets or percentiles.

Syntax

NTILE(n) Over (Order by column)

→ n: The no. of groups to devide the rows into.

→ Order by column: Specifies the order in which rows are assigned to groups.

How it works:

- **Dynamic Row Grouping**
  Unlike static row filtering, NTILE(n) allows you to partitions the data dynamically into [n] groups.

- **Even Distribution**
  If rows are evenly divisible by [n], each group has the same no. of rows.

- **Handling Uneven Rows**
  When rows are not evenly divisible, some groups get one more row than other.

- **Independent of Field values**
  NTILE assigns row numbered based purely on count, not the actual field values.

- Row with the same value in the Order by column may end up in different groups.

When to use it

- Use NTILE when you want to divide rows into equal-sized groups (e.g. percentiles, deciles, quartiles).

- Example: Divide product into price buckets or rank customers into spending tiers.

tablename: vendor_inventory2

vendor_id	market_date	product_id	original_price
1	2023-10-01	101	50.00
1	2023-10-02	102	40.00
1	2023-10-03	103	30.00
1	2023-10-04	104	60.00
2	2023-10-01	201	55.00
2	2023-10-02	202	45.00
2	2023-10-03	203	35.00
2	2023-10-04	204	70.00
3	2023-10-01	301	65.00
3	2023-10-02	302	66.00
3	2023-10-03	303	55.00
3	2023-10-04	304	25.00
Int	Date	Int	Decimal (10,2)

20.00
60.00
55.00
45.00
35.00
70.00

1. Using NTILE to divide Data into 4 Groups (Quartile)

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
```

PAGE NO.....
DATE / /

2. Using NTILE with Partitioning into 2 groups

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
    NTILE(2) OVER (PARTITION BY vendor_id ORDER BY
        original_price DESC) As price_group
FROM vendor_inventory2
ORDER BY vendor_id, original_price Desc;
```

vendor_id	market_date	product_id	original_price	price_group
1	2023-10-04	104	20.00	2
2	2023-10-01	201	60.00	1
2	2023-10-02	202	53.00	1
2	2023-10-03	203	43.00	2
2	2023-10-04	204	33.00	2
3	2023-10-01	301	70.00	1
3	2023-10-02	302	65.00	1
3	2023-10-03	303	55.00	2
3	2023-10-04	304	25.00	2

Explanation

- The rows are divided into 2 groups (price_group) for each vendor.
- The first group contains the more expensive product for each vendor.
- The second group contains the less expensive product for each vendor.

PAGE NO.....
DATE / /

Aggregate Window Functions

1. What They DO:

→ Aggregate window functions perform calculations across a set of rows related to the current row (a 'window').

→ Unlike regular aggregate functions (e.g. SUM(), AVG()) they do not collapse rows into a single result. Instead, they return the aggregate value for each row in partition.

2. Syntax

AGGREGATE_FUNCTION (column) OVER (PARTITION BY partition_column ORDER BY sort_column)

→ AGGREGATE_FUNCTION: Function like SUM(), AVG(), COUNT() etc.

→ PARTITION BY: Divides the data into groups (partitions)

→ ORDER BY: Specifies the order of rows within each partitions.

3) Common Use Cases:

→ Comparing individual values to group averages or totals.
→ Calculating running totals or cumulative sums
→ Counting items within partitions.

PAGE NO.....
DATE / /

1. Using AVG() as a Window function.

```
SELECT
    vendor_id,
    market_date,
    product_id,
    original_price,
    AVG(original_price) OVER (PARTITION BY market_date
        AS avg_price_by_market_date
From vendor_inventory3.
Order by market_date, original_price Desc;
```

Partition For '2023-10-01'

Average for 2023-10-01 = $\frac{60+40+60+53+70+65}{6} = 56.67$

Partition For '2023-10-02'

Average for 2023-10-02 = $\frac{30+20+45+36+53+25}{6} = 35.00$

output

vendor_id	market_date	product_id	original_price	avg_price_by_market_date
3	2023-10-01	301	70.00	56.66
3	2023-10-01	302	65.00	
2	2023-10-01	202	55.00	
1	2023-10-01	101	50.00	
1	2023-10-01	102	40.00	
3	11 11-02	303		
2	11 11-02	203		
2	11 11-02	204		
1	11 11-02	103		
3	11 11-02	304		
1	11 11-02	104		

output: table_name = ~~inventory~~ vendor_inventory_3.

vendor_id	market_date	product_id	original_price	avg_price_by_market_date
3	2023-10-01	301	70.00	56.667
3	2023-10-01	302	65.00	56.667
2	2023-10-01	201	60.00	"
2	2023-10-01	202	55.00	"
1	2023-10-01	101	50.00	"
1	2023-10-01	102	40.00	"
3	2023-10-02	303	55.00	33.00
2	2023-10-02	203	45.00	33.00
2	2023-10-02	204	35.00	"
1	2023-10-02	103	30.00	"
3	2023-10-02	304	25.00	"
1	2023-10-02	104	20.00	"

Created table

vendor_id	market_date	product_id	original_price	~~avg_price_by~~ market_date
1	2023-10-01	101	50.00	
1	2023-10-01	102	40.00	
1	2023-10-02	103	30.00	
1	2023-10-02	104	20.00	
2	2023-10-01	201	60.00	
2	2023-10-02	202	53.00	
2	2023-10-02	203	45.00	
2	2023-10-02	204	35.00	
3	2023-10-01	301	70.00	
3	2023-10-01	302	65.00	
3	2023-10-02	303	55.00	
3	2023-10-02	304	25.00	

PAGE NO.....
DATE / /

2. Using Count() as a window Functions

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
Count(product_id) Over ( Partition by market_date, vendor_id)
                    As vendor_product_count.
From vendor_inventory3
Order by vendor_id, market_date, original_price desc;
```

- Partition : `Partition by market_date, vendor_id`

→ divides the rows into groups based on each combination of `market_date` and `vendor_id`

→ Each group (or partition) consist of rows with the same `market_date` and the same `vendor_id`

- Counting : Count (product_id).
  → Counts the no. of products `(product_id)` in each partition

Partition `vendor_id = 1`, `market_date = '2023-10-01'`

vendor_id	market_date	product_id	original_price	vendor_product_count
1	2023-10-01	L01	50.00	2
1	2023-10-01	L02	40.00	2

- Since there are 2 product (L01 and L02) for vendor 1 on `2023-10-01`, the `COUNT()` function returns 2 for each row.

PAGE NO.....
DATE / /

Why doesn't Count(product_id) Only Count a Specific product_id?

→ When you use COUNT(product_id) over (partition by market_date, vendor_id), you are counting the no. of rows (product) within each partitions (group), regardless of the specific product ID value.

How count() Works Here:

→ It doesn't count only product_id = 101 or 102 - instead, it counts all the product IDs within each partitions defined by Partition by market_date, vendor_id.

Why Doesn't it Count Just the Row's Product?

The Count() function is an aggregate window function

→ It looks at all rows in the same partition and return a count.

→ It doesn't just focus on a single row or product_id

To count specific product_id.

```
Count( Case when product_id = 101 Then 1 End) Over
    ( Partition by market_date, vendor_id ) As
                                count_of_product_101
```

PAGE NO.....
DATE / /

output

vendor_id	market_date	product_id	original	vendor_product_count.
1	2023-10-01	101	50.00	2
1	2023-10-01	102	40.00	2
1	2023-10-02	103	30.00	2
1	2023-10-02	104	20.00	2
2	2023-10-01	201	60.00	2
2	2023-10-01	202	55.00	2
2	2023-10-02	203	45.00	2
2	2023-10-02	204	35.00	2
3	2023-10-01	301	70.00	2
3	2023-10-01	302	65.00	2
3	2023-10-02	303	55.00	2
3	2023-10-02	304	25.00	2

3) Using Sum() as a Window function:

```
Select
    vendor_id,
    market_date,
    product_id,
    original_price,
SUM(original_price) Over( Partition by vendor_id Order by
                          market_date, product_id) AS
                          running_total
From vendor_inventory3.
Order by vendor_id, market_date, product_id;
```

Explanations

For vendor_id = 1

Running total (sum() applied) = 50.00 & 50 + 40 = 90, 90 + 30 = 120, 120 + 20 = 140

PAGE NO.....
DATE / /

output

vendor_id	market_date	product_id	original_price	running_total
1	2023-10-01	101	50.00	50.00
1	2023-10-01	102	40.00	90.00
1	2023-10-02	103	30.00	120.00
1	2023-10-02	104	20.00	140.00
2	2023-10-01	201	60.00	60.00
2	2023-10-02	203	43.00	103.00
2	2023-10-02	204	35.00	195.00
3	2023-10-01	301	70.00	70.00
3	2023-10-01	302	65.00	135.00
3	2023-10-02	303	55.00	190.00
3	2023-10-02	304	23.00	213.00

LAG and LEAD.

PAGE NO.....
DATE / /

1. LAG function

- Retrieves the value from a previous row in the dataset.
- Syntax: LAG (column, offset) over (Partition By partion_column ORDER BY sort-column)
- column: The column to retrieve the value from...
- offset: The no. of rows back to look (default is 1).
- Partition By: Divides the data into groups (optional).
- ORDER By: Specifies the order of rows within each partition.

2. LEAD Function

- Retrieves the value from a subsequent row in the dataset.
- Syntax

LEAD (column, offset) over (Partition by partition_column Order by sort-column).

- column: The column to retrieve the value from.
- offset: The no. of rows forward to look (default 1).
- Partition by: Divides the data into groups (optional)
- Order by: Specifies the order of rows within each partition.

PAGE NO.....
DATE / /

3. Use Cases:

-> Compute current values with previous or next values (e.g. sales trends, booth assignments).

-> Calculate changes over time (e.g. week-to-week sales changes).

-> Identify pattern or anomalies in sequential data.

table name: vendor_booth_assignments.

market_date	vendor_id	booth_number
2023-10-01	1	101
2023-10-01	2	102
2023-10-01	3	103
2023-10-08	1	102
2023-10-08	2	101
2023-10-08	3	103
2023-10-15	1	101
2023-10-13	2	102
2023-10-13	3	104

Date | Int | Int

PAGE NO.....
DATE / /

table name: ~~vendor_booth_assignments~~ employee_salaries

employee_id	salary_date	salary
1	2024-01-01	50000.00
1	2024-02-01	52000.00
1	2024-03-01	51000.00
1	2024-04-01	53000.00
2	2024-01-01	60000.00
2	2024-02-01	62000.00
2	2024-03-01	61000.00
2	2024-04-01	63000.00

Int Date Decimal (10,2)

(1) Use Lag() to compare previous salaries.

```
Select
    employee_id,
    Salary_date,
    salary,
LAG(salary, 1) OVER (Partition by employee_id Order by
                     salary_date) As previous_salary.
From employee_salaries.
Order by employee_id, salary_date;
```

How Lag() works in this Query:

1. Partition by employee_id

→ It ensures that the Lag() function only considers previous salaries for the same employee.

PAGE NO.....
DATE / /

2) Order by salary_date.

→ It orders salaries chronologically within each employee's partitions, ensuring that the previous salary is from the correct past date.

3) LAG(salary, 1)

→ It fetches the previous row's salary for the same employee.

4). ORDER By employee_id, salary_date.

→ The final ordering ensures the output is structured by employee and date.

Output

employee_id	salary_date	salary	previous_salary
1	2024-01-01	50000.00	Null
1	2024-02-01	52000.00	50000.00
1	2024-03-01	51000.00	52000.00
1	2024-04-01	53000.00	51000.00
2	2024-01-01	60000.00	Null
2	2024-02-01	62000.00	60000.00
2	2024-03-01	61000.00	62000.00
2	2024-04-01	63000.00	61000.00

- The first row has Null because there is no previous salary.
- The salary difference is visible by comparing salary and previous_salary

PAGE NO.....
DATE / /

(8) Use LEAD() to see future salaries.

```
Select
    employee_id,
    salary_date,
    salary,
LEAD(salary, 1) Over(Partition by employee employee_id
                Order by salary_date) As next_salary
From employee_salaries
Order by employee_id, salary_date;
```

output

employee_id	salary_date	salary	next_salary
1	2024-01-01	50000	52000
1	2024-02-01	52000	54000
1	2024-03-01	54000	53000
1	2024-04-01	53000	Null
2	2024-01-01	60000	62000
2	2024-02-01	62000	61000
2	2024-03-01	61000	63000
2	2024-04-01	63000	Null